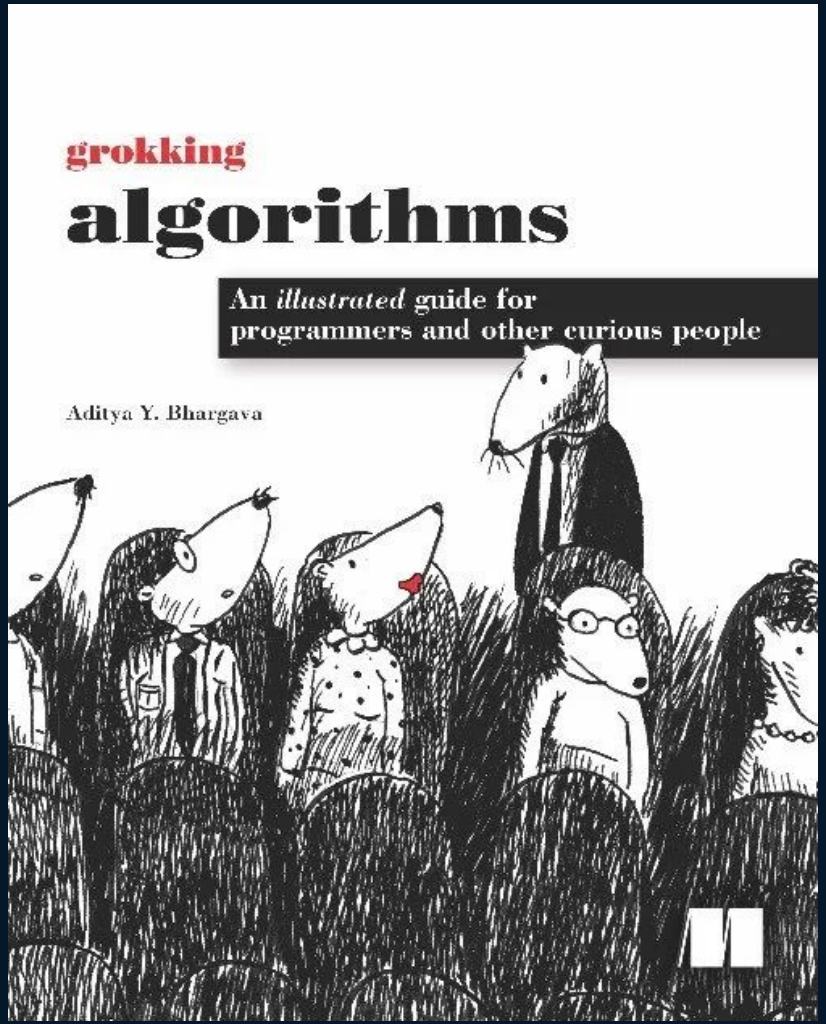
Algorithms

Intro

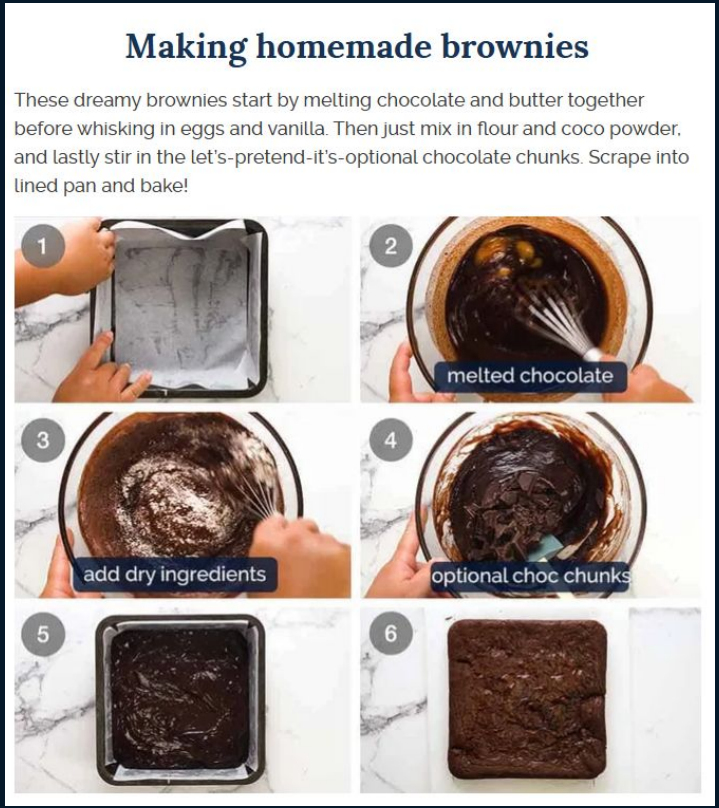

grokking
algorithms
An illustrated guide for
programmers and other curious people
Aditya Y. Bhargava

Algorithm

- a precise set of instructions for performing a specific task

Algorithm

- a precise set of instructions for performing a specific task



https://www.recipetineats.com/easy-chocolate-brownies/, access: 16.02

Big O notation

- Big O notation is a mathematical notation that describes the approximate size of a function on a domain.

- We want to describe the worst case scenario (longest time of execution)
- Sometimes we need average time of execution (later in book; not to worry now)
- we can meet two concepts: time complexity and space complexity (related to memory usage)

Big O notation - most common

- O(1) - constant
- O(log n) - logarithmic - e.g. binary search
- O(n) - linear - e.g. linear search (sequential search)
- O(n * log n) - log-linear, e.g. merge sort, quick sort (average case)
- O(n^2) - quadratic - e.g. selection sort, quick sort (worst case)
- O(n!) - factorial - e.g. Travelling salesman problem

(examples based on time complexity)

- https://medium.com/@teamtechsis/big-o-notation-explained-in-plain-english-983b0f7227aa, access: 18.02.2026,
- https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-merge-sort/, access: 18.02.2026,
- https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-quick-sort/, access: 18.02.2026,

BIG O NOTATION

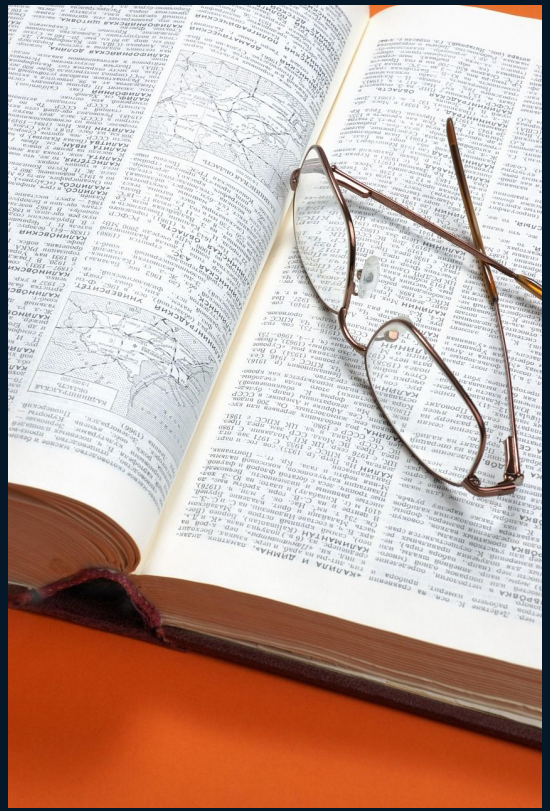
- O(1) = O(yeah)
- O(log n) = O(nice)
- O(n) = O(ok)
- O(n^2) = O(my)
- O(2^n) = O(no)
- O(n!) = O(mg)

Binary search

- list **must be sorted**
- instead of doing linear search (nums[0], nums[1] …. nums[n-2], nums[n-1]) with 0(n) complexity
- we see what’s in the middle and how it relates to our target
  - if the target is smaller, we discard the right half and repeat on the left
  - if the target is larger, we discard the left half and repeat on the right
  - if it matches, we're done

Binary search

If we search for “algorithms” linear search seems faster than eg. looking for “jam”, “oysters”, or “zoo”.

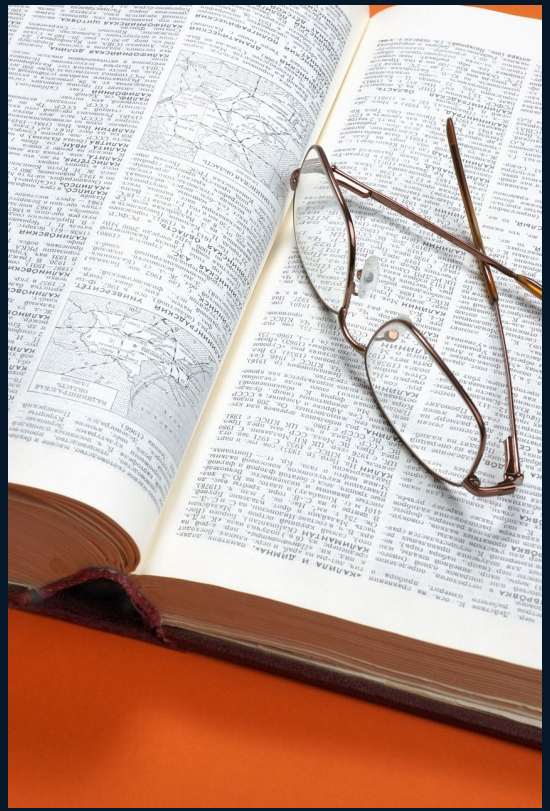


Binary search

If we search for “algorithms” linear search seems faster…

…but we need to know average time for average input.

(testing times for “algorithms”, “jam”, “oysters”, “zoo” and many more)

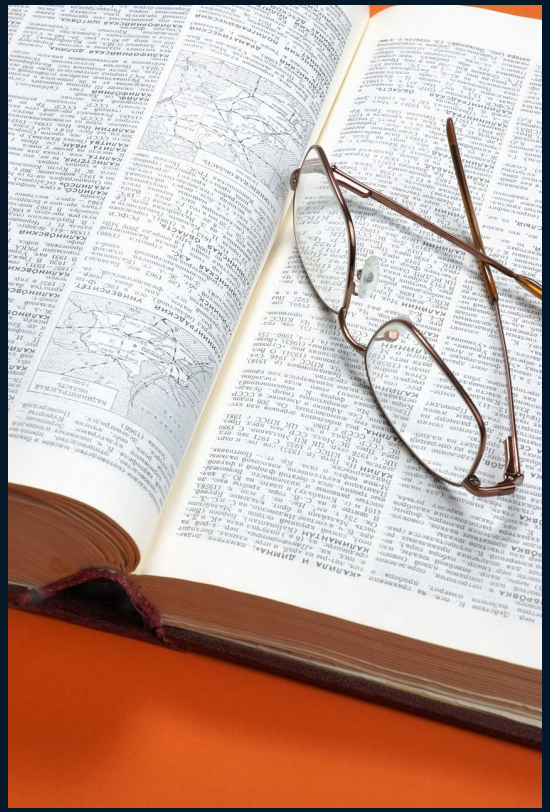


Binary search

Optimal approach: see what's in the middle O_O

for 8 elements:

- linear - 8
- binary - $\log_2 8 = 3$



Binary search

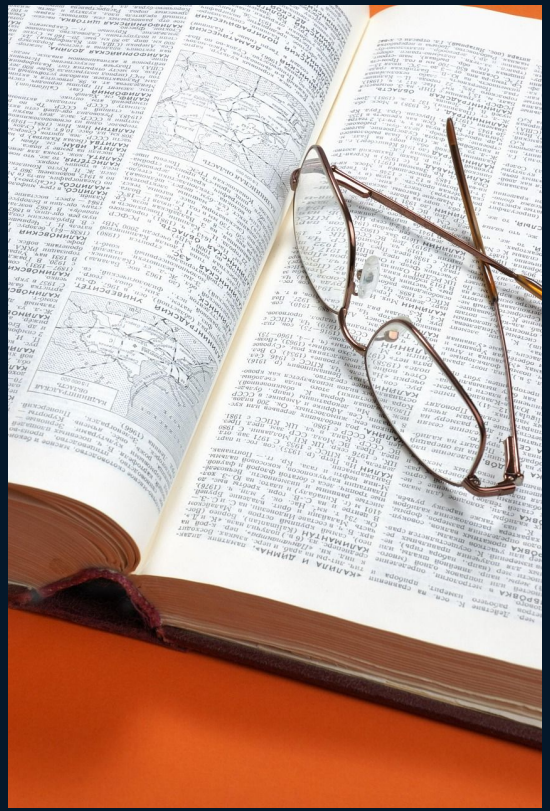
Optimal approach: see what's in the middle O_O

for 8 elements:

- linear - 8
- binary - $\log_2 8 = 3$

for 2048 elements:

- linear - 2048
- binary - $\log_2 2048 = 11$



Let’s talk about... Java drawers

https://pixabay.com/pl/photos/pud%c5%82a-szuflady-skrzynki-pocztowe-1834406/

Arrays, ArrayLists, vs. Linked Lists (Java)

Arrays:

```
// Declaring and initializing an array of integers

int[] numbers = new int[5]; // Array of size 5

numbers[0] = 10;

numbers[1] = 20;

numbers[2] = 30;

numbers[3] = 40;

numbers[4] = 50;
```

Characteristics:

- fixed size
- index-based access
- contiguous memory (all elements stored together in memory)

Advantages: fast access - O(1)

Limitations:

- fixed size
- insertion, deletion - O(n)

Based on: https://medium.com/@ashishkumarjena1437/arrays-linked-lists-and-arraylists-understanding-the-core-differences-6ee11d47cf1d

Arrays, ArrayLists, vs. Linked Lists (Java)

ArrayList - resizable array:

```
ArrayList<String> fruits = new ArrayList<>();

    // Adding elements

    fruits.add("Apple");

    fruits.add("Banana");

    fruits.add("Cherry");
```

Characteristics:

- dynamic resizing
- index-based access
- non-primitive storage (String, Integer)

Advantages:

- dynamic size
- fast access - O(1)

Limitations:

- resizing overhead

Based on: https://medium.com/@ashishkumarjena1437/arrays-linked-lists-and-arraylists-understanding-the-core-differences-6ee11d47cf1d

Arrays, ArrayLists, vs. Linked Lists (Java)

LinkedList:

```
LinkedList<Integer> list = new LinkedList<>();

// Adding elements

list.add(10);

list.add(20);

list.add(30);
```

Characteristics:

- dynamic size
- non-contiguous memory (data scattered)
- sequential access

Advantages:

- flexible size
- efficient insertions and deletions - O(1)

Limitations:

- slow access - O(n)

Based on: https://medium.com/@ashishkumarjena1437/arrays-linked-lists-and-arraylists-understanding-the-core-differences-6ee11d47cf1d

Arrays (lists) vs. Linked Lists

Feature	Array	Linked List	ArrayList
Size	Fixed	Dynamic	Dynamic
Memory Allocation	Contiguous	Non-contiguous	Contiguous (resizable)
Access Time	O(1)	O(n)	O(1)
Insertion/Deletion	O(n)	O(1) (at head)	O(n)
Memory Efficiency	High (minimal overhead)	Extra memory for pointers	Can waste memory during resizing
Resizing	Not possible	Easy	Automatic resizing
Use Case	Fixed data size	Frequent insertions/deletions	Dynamic data size, fast access

Based on: https://medium.com/@ashishkumarjena1437/arrays-linked-lists-and-arraylists-understanding-the-core-differences-6ee11d47cf1d